

RESUMO DO PROJETO

Estrutura da apresentação do projeto

Instruções gerais do projeto:

O projeto deve estar completamente no GITHUB.

A equipe deve preparar uma apresentação com **13** slides, e enviar o PDF dos slides para o professor no classroom.

Algumas equipes vão ser sorteadas para apresentar no último dia de aula.

Todas as equipes vão ser avaliadas de acordo com o projeto.

Tempo de apresentação **máximo** 15 minutos. Por favor, treine para que sua apresentação se adeque ao tempo.

ATENÇÃO: o projeto não precisa estar “finalizado”, no sentido de ter implementado rigorosamente todos os requisitos funcionais e não funcionais. Ele deve ser uma versão que contém os principais blocos implementados. Trata-se de uma “maquete” funcional, que pode ser melhorada futuramente.

A apresentação não deve se transformar em uma revisão teórica da disciplina. **Cada slide deve mostrar decisões e evidências do próprio projeto do grupo**, e não explicar genericamente o que é Git, PyTorch, arquitetura, requisitos ou tipagem.

O nome dos membros da equipe deve estar no slide 1, e apenas um membro precisa enviar como tarefa no classroom o slide da turma.

Abaixo, apresento o que deve conter em cada um dos 13 slides.

1. Resumo do projeto

Slide 1 — Problema, motivação e objetivo

Apresentar de forma muito objetiva:

- nome do projeto;
- membros da equipe (colocar nomes completo para fácil identificação)
- problema que está sendo resolvido;
- motivação para escolha do problema;
- público-alvo ou contexto de utilização;
- objetivo principal da aplicação;
- qual papel a IA desempenha na solução;
- link para o repositório no GITHUB.

O slide deve responder essencialmente:

Qual problema estamos tentando resolver e por que ele é relevante?

Slide 2 — Dados e solução proposta

Apresentar:

- base de dados utilizada;
 - origem dos dados;
 - quantidade aproximada de amostras e atributos;
 - variável que será prevista/classificada;
 - entrada e saída do sistema;
 - descrição resumida da solução proposta.
-

2. Requisitos

Slide 3 — Requisitos funcionais

Apresentar os **principais requisitos funcionais**, preferencialmente entre 5 e 8. Os requisitos devem ter surgido a partir da tarefa realizada sobre Engenharia de Requisitos.

Por exemplo:

- **RF01** — O sistema deve carregar a base de dados.
- **RF02** — O sistema deve realizar o pré-processamento dos dados.
- **RF03** — O sistema deve treinar o modelo de IA.
- **RF04** — O sistema deve avaliar o modelo.
- **RF05** — O sistema deve realizar previsões para novos dados.
- **RF06** — O sistema deve salvar e carregar o modelo treinado.

Os requisitos devem ser **específicos para o projeto do grupo**, e não simplesmente copiar esses exemplos.

Slide 4 — Requisitos não funcionais e critérios de aceitação

Apresentar os principais requisitos relacionados à qualidade do sistema, como:

- desempenho;
- modularidade;
- testabilidade;
- reprodutibilidade;
- manutenibilidade;
- segurança, quando aplicável;
- tempo de resposta, quando relevante.

Para os requisitos mais importantes, apresentar **como será verificado se o requisito foi atendido**.

Exemplo:

RNF01: O modelo deve obter F1-score $\geq 0,80$ no conjunto de teste.

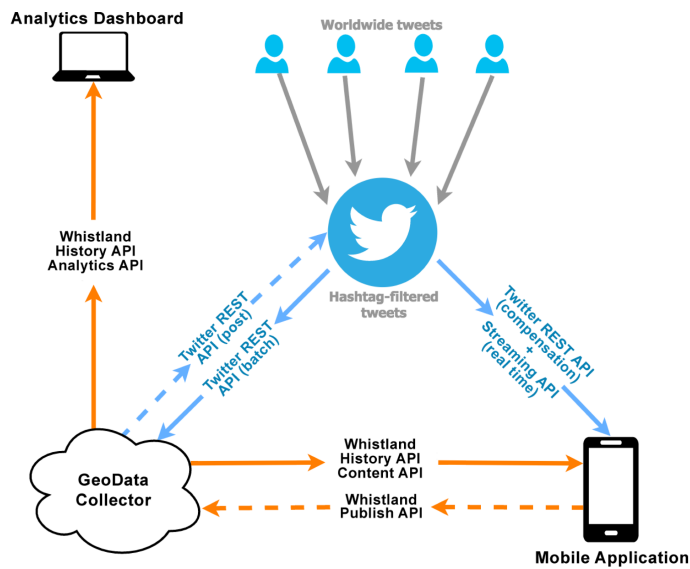
Critério de aceitação: execução da avaliação final no conjunto de teste.

3. Arquitetura do sistema

Slide 5 — Diagrama da arquitetura

Apresentar obrigatoriamente um **diagrama da arquitetura real do projeto**.

Por exemplo:



O diagrama deve mostrar os **principais componentes do sistema e suas relações**.

Slide 6 — Decisões arquiteturais

Explicar as principais decisões tomadas:

- responsabilidade de cada componente;
- fluxo dos dados;
- onde ocorre o pré-processamento;
- onde ocorre treinamento;
- onde ocorre inferência;
- onde o modelo é armazenado;
- como os componentes se comunicam;
- por que essa arquitetura foi escolhida.

O grupo deve relacionar suas decisões, quando pertinente, a conceitos como: **separação de responsabilidades, alta coesão, baixo acoplamento, manutenibilidade e testabilidade**.

4. Modularização — organização das pastas

Slide 7 — Estrutura do projeto

Mostrar a **estrutura real do repositório**.

Por exemplo:

project/

├── data/

├── docs/

├── src/

| ├── data/

| ├── preprocessing/

| ├── models/

| ├── training/

| ├── evaluation/

| └── inference/

└── tests/

└── requirements.txt

└── README.md

└── main.py

Não basta mostrar a árvore. O grupo deve explicar rapidamente **por que o código foi separado dessa maneira**.

Slide 8 — Responsabilidades dos módulos

Escolher os principais módulos e explicar suas responsabilidades.

Por exemplo:

Módulo	Responsabilidade
data	carregamento dos dados

preprocessing	preparação e transformação
models	definição do modelo
training	treinamento
evaluation	cálculo das métricas
inference	predição de novos exemplos
tests	testes automatizados

Mostrar também como o `main.py` integra esses componentes.

5. Exemplos de código com tipagem e PyTorch

Evite encher os slides de código. Não precisa colocar todos os códigos. O objetivo é seleccionar **pequenos exemplos representativos da qualidade da implementação**.

Slide 9 — Código Python, NumPy e tipagem

Mostrar 2 ou 3 funções importantes do projeto.

Por exemplo:

```
def load_data(path: str) -> pd.DataFrame:
```

```
...
```

```
def standardize(X: np.ndarray) -> np.ndarray:
```

```
...
```

```
def split_data(
```

```
X: np.ndarray,  
y: np.ndarray  
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:  
  
...
```

Explicar:

- entradas;
- saídas;
- type hints;
- utilização de NumPy;
- responsabilidade da função.

O objetivo é demonstrar que o código possui **interfaces claras e responsabilidades bem definidas**.

Slide 10 — Implementação do modelo de IA em PyTorch

Mostrar um pequeno trecho do modelo real:

```
class NeuralModel(nn.Module):
```

```
    def __init__(  
        self,  
        input_dim: int,  
        hidden_dim: int,  
        output_dim: int  
    ):  
        super().__init__()  
        ...
```

O slide deve identificar:

- arquitetura da rede neural ou do modelo de IA utilizado;
- função de perda;
- otimizador;
- número de épocas;
- batch size;
- CPU/GPU;

- salvamento e carregamento do modelo.
-

6. Experimentos realizados

Slide 11 — Configuração experimental

Explicar rapidamente **como o modelo foi avaliado**.

Apresentar:

- divisão treino/teste ou treino/validação/teste;
- métricas utilizadas;
- quantidade de épocas;
- hiperparâmetros;
- diferentes configurações testadas;

O grupo deve dizer **o que estava tentando descobrir com os experimentos**.

7. Testes automatizados — 1 slide

Slide 12 — Exemplos de testes e evidências

O grupo deverá mostrar **testes reais do projeto**, e não apenas explicar conceitualmente o que é teste unitário.

Selecionar **2 ou 3 exemplos relevantes**, como:

- teste do carregamento dos dados;
- teste do pré-processamento;
- teste de **shape** dos tensores;
- teste da saída do modelo;
- teste de salvamento/carregamento;
- teste de alguma regra específica do sistema.

Por exemplo:

```
class TestPreprocessing(unittest.TestCase):
```

```
    def test_standardize_shape(self):
```

```
        ...
```

```
    def test_standardize_mean(self):
```

...

O slide deve mostrar também a **evidência da execução do teste**:

Ran 15 tests

OK

Slide 13 — Resultados, conclusão e limitações

Apresentar os principais resultados em **gráfico ou tabela**:

- desempenho em treinamento;
- desempenho em teste;
- métricas relevantes;
- comparação das configurações;

E responder:

O modelo proposto resolveu satisfatoriamente o problema?

Os requisitos definidos no início foram atendidos?

Quais são as principais limitações?

Quais os próximos passos.

Finalizar com 2 ou 3 possibilidades de melhoria futura.

Em síntese

A apresentação passa a contar uma história muito mais direta:

- 1. Resumo:** *Qual problema queremos resolver?*
- 2. Requisitos:** *O que o sistema precisa fazer?*
- 3. Arquitetura:** *Como projetamos a solução?*
- 4. Modularização:** *Como organizamos essa solução no código?*
- 5. Código/PyTorch:** *Como implementamos?*
- 6. Experimentos:** *Funcionou? Quais evidências temos?*